

DM - Greffes génétiquement modifiées d'un arbre binaire

NOM:LIU PRENOM:Xuanshuo

arbres_binaires.c / greff.c 50% / main.c 50%

NOM:CAO PRENOM:Yunliang

saage.c / greff.c 50% / main.c 50%

1.greffe.c

Liste des fonctions

int copie(Arbre *dest, Arbre source)

Fonction : Copie de l'arbre source dans l'arbre de destination.

Paramètres :

dest : Pointeur sur pointeur de l'arbre de destination.

source : Arbre source.

Renvoie : 1 si la copie réussit, sinon 0.

void insertSubtree(Arbre *node, Arbre subtree)

Fonction : Insertion d'un sous-arbre dans un nœud d'arbre.

Paramètres :

node : Pointeur sur pointeur du nœud de l'arbre cible.

subtree : Sous-arbre à insérer.

Renvoie : Rien.

void expansion(Arbre *A, Arbre B)

Fonction : Expansion de l'arbre A par un autre arbre B.

Paramètres :

A : Pointeur sur pointeur de l'arbre à étendre.

B : Arbre à utiliser pour l'expansion.

Renvoie : Rien.

Détails des fonctions :

int copie(Arbre *dest, Arbre source)

Cette fonction copie l'arbre source dans l'arbre de destination.

Algorithme :

Si l'arbre source est vide, définir l'arbre de destination comme vide et renvoyer 1.

Sinon, allouer de la mémoire pour l'arbre de destination et copier la valeur de l'arbre source.

Si l'allocation de mémoire échoue, renvoyer 0.

Copier récursivement les sous-arbres gauche et droit de l'arbre source.

En cas d'échec de la copie, libérer la mémoire allouée et renvoyer 0.

Renvoyer 1 en cas de succès de la copie.

void insertSubtree(Arbre *node, Arbre subtree)

Cette fonction insère un sous-arbre dans un nœud vide de l'arbre cible.

Algorithme

Si le nœud cible ou le sous-arbre est vide, retourner.

Si le sous-arbre gauche du nœud cible est vide, copier le sous-arbre dans le sous-arbre gauche.

Sinon, chercher récursivement un nœud vide dans le sous-arbre gauche et insérer le sous-arbre.

Si le sous-arbre droit du nœud cible est vide, copier le sous-arbre dans le sous-arbre droit.

Sinon, chercher récursivement un nœud vide dans le sous-arbre droit et insérer le sous-arbre.

void expansion(Arbre *A, Arbre B)

Cette fonction étend l'arbre A à l'aide d'un autre arbre B.

Algorithme

Si l'arbre A est vide, retourner.

Parcourir récursivement les sous-arbres gauche et droit de l'arbre A.

Si la valeur du nœud correspond à la racine de l'arbre B, effectuer l'expansion :

Copier l'arbre B.

Insérer l'arbre A dans le nouvel arbre copié.

Mettre à jour le pointeur de l'arbre A vers le nouvel arbre.

Renvoyer.

Remarques

Toutes les fonctions supposent des paramètres valides, c'est-à-dire non NULL.

Lors de l'utilisation de la fonction `expansion()`, assurez-vous que l'arbre donné et l'arbre cible ont le même type de valeur de nœud.

greffe.c

Fonctions :

Noeud *alloue_noeud(char *s)

Cette fonction alloue dynamiquement la mémoire pour un nœud de l'arbre et initialise ses valeurs.

Paramètres :

s : La valeur à stocker dans le nœud.

Retour : Un pointeur vers le nœud alloué, ou NULL en cas d'échec de l'allocation.

void librer(Arbre *A)

Cette fonction libère récursivement la mémoire allouée pour un arbre binaire.

Paramètres :

A : Pointeur vers la racine de l'arbre à libérer.

Retour : Aucun.

int construit_arbre(Arbre *a, char **input)

Cette fonction construit un arbre binaire à partir d'une chaîne de caractères.

Paramètres :

a : Pointeur vers la racine de l'arbre à construire.

input : Pointeur vers la chaîne de caractères à partir de laquelle l'arbre doit être construit.

Retour : 1 en cas de succès, 0 en cas d'échec.

saage.c

Fonctions:

int serialise_aux(FILE *f, Arbre A, int depth)

Cette fonction réalise la sérialisation récursive d'un arbre.

Paramètres :

f : Pointeur vers le fichier où l'arbre doit être sauvegardé.

A : Pointeur vers la racine de l'arbre à sérialiser.

depth : Profondeur actuelle dans l'arbre (pour l'indentation).

Retour : 1 en cas de succès, 0 en cas d'échec.

int serialise(char *nom_de_fichier, Arbre A)

Cette fonction sauvegarde un arbre dans un fichier.

Paramètres :

nom_de_fichier : Nom du fichier où sauvegarder l'arbre.

A : Pointeur vers la racine de l'arbre à sauvegarder.

Retour : 1 en cas de succès, 0 en cas d'échec.

int deserialise_aux(FILE* f, Arbre* a)

Cette fonction réalise la désérialisation récursive d'un arbre.

Paramètres :

f : Pointeur vers le fichier à partir duquel l'arbre doit être reconstruit.

a : Pointeur vers le pointeur de la racine de l'arbre à reconstruire.

Retour : 1 en cas de succès, 0 en cas d'échec.

int deserialise(char* nom_de_fichier, Arbre* a)

Cette fonction reconstruit un arbre à partir d'un fichier.

Paramètres :

nom_de_fichier : Nom du fichier à partir duquel reconstruire l'arbre.

a : Pointeur vers le pointeur de la racine de l'arbre reconstruit.

Retour : 1 en cas de succès, 0 en cas d'échec.

Mode d'utilisation:

- saage -E fichier.saage crée une sauvegarde dans le fichier .saage d'un arbre saisi par l'utilisateur au clavier.
- saage -G s.saage g.saage crée l'arbre où le greffon g.saage est appliqué à l'arbre source stocké dans s.saage. Le résultat de la greffe sera fournis sur la sortie standard au format .saage
- saage -V fichier.saage dot.dot. Pour améliorer la réalisation du DM, nous avons utilisé les fonctions du TP4 pour visualiser l'arbre binaire. Bien sûr, l'ordinateur doit pouvoir utiliser Envice et les fichiers Dot.

Conclusion:

La partie la plus difficile a été la fonction expansion() dans le code C, chargée d'implémenter l'opération d'expansion de l'arbre. Après plusieurs essais et erreurs, nous avons constaté

que cette fonction était difficile à implémenter directement. Par conséquent, nous avons décidé d'utiliser une autre fonction comme aide. En utilisant la fonction `insertSubtree()` comme support, nous avons réussi à résoudre le problème d'expansion de l'arbre tout en préservant la clarté et la maintenabilité du code. Cette décision nous a permis de traiter plus efficacement les opérations sur les arbres et de garantir la précision et la fiabilité du code.